

Project 2: Toxic Comment Classification

Objective

This project is a continuation of the first week's assignment on toxic comment detection. You will **redo the problem set using modern NLP tools**: word embeddings, transformers, and prompt engineering. The goal is to understand what changes (in performance, workflow, interpretability) when we move from bag-of-words + classical ML to contemporary approaches.

Grading

You will deliver:

1. **Report**: PDF, max 2 pages; compare Project 1 and Project 2 (figures/tables required). Please don't forget to write down the names of all the people in your group on it. Send it to me with the following name: *Name1Name2Name3Name4_MLnewdata_Project_Session2*
2. **Notebook**: .py; reproducible with a fixed random state.
3. **8 minutes oral presentation** highlighting the comparison.

Use of external libraries and LLMs (e.g. Hugging Face, OpenAI, ChatGPT):

- You are expected to use modern libraries.
- State any AI/tool assistance you used and how.

Dataset

Same as Project 1: labelled Wikipedia comments with a binary label: toxic (1) or not toxic (0). You may reuse your preprocessing pipeline, but adapt it where embeddings/transformers make some steps redundant.

Tasks

1. **From Project 1 to Project 2.** Briefly recall your best bag-of-words + classical ML model from Project 1. What were its strengths and weaknesses? Keep this as your baseline.
2. **Word embeddings.**
 - Use pretrained word embeddings (e.g. GloVe, fastText) or sentence embeddings (e.g. MiniLM).
 - Train a simple classifier (Logistic Regression or SVM) on these embeddings to classify comments as toxic or non-toxic using this approach
3. **Transformers.**
 - Use a transformer-based model. You may fine-tune or use Hugging Face pipelines.
 - Classify the comments as toxic or non-toxic using this approach
4. **Prompt engineering.**
 - Query a large language model of your choice
 - Design at least three prompts for toxic comment detection (e.g. zero-shot classification, role prompting, few-shot)
 - Classify the comments as toxic or non-toxic using this approach
5. **Evaluation:** for each approach:
 - Produce a classification report (precision, recall, F1)
 - Calculate the area under the curve
6. **Reflection**

- **Cost:** For each approach, report the time it takes to compute the classification. How might this affect deployment choices?
- **Bias:** Imagine the embedding model flags a neutral sentence like “She is a woman” as toxic. Why could this happen ? What does it reveal about the risks of using embeddings or transformers for moderation?
- **Transparency:** Suppose a user asks why a comment was flagged as toxic. Which model is easiest to explain why and which is hardest? Give one concrete reason for each.
- **Privacy:** If you use an external LLM API for prompting, what happens to the user comments you send? What risks does this create for Wikipedia?